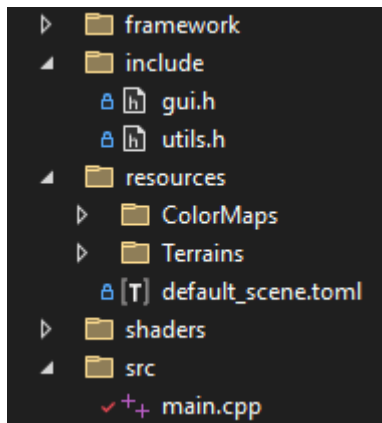


3D Data Visualizer Program Structure

I. Program Structure

A. File and Folder Structure

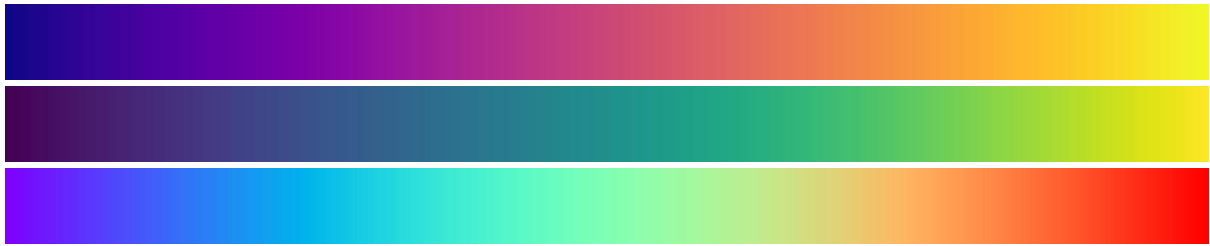


- “*framework*”: folder containing the framework code, taken from the template.
- “*include*”: folder containing header files with helper methods and variables.
- “*resources*”: folder containing the program resources:
 - “*default_scene.toml*”: file describing the default program configuration.
 - “*Terrains*”: folder containing image files to use for the 3D visualization.
 - “*ColorMaps*”: folder containing the color maps that can be used to visualize the data.
- “*shaders*”: folder containing all the GLSL shader files used by this program.
- “*src*”: folder containing the main.cpp file, the code of the code.

B. Resource Files Type and Format:

1. **Color maps:** they should be in PNG format, with either 8bits or 16bits RGBA. The program loader automatically detects the resolution. During the loading, it converts 8bit images into 16bit images such that everything is stored as 16bits images in the program. The following color maps are available:





In order: magma, inferno, plasma, viridis and rainbow color maps.

All those gradients are luminant gradients, except the rainbow, which is used to illustrate the shortcomings of using rainbow gradients to visualize data. Those files are 2048x128 PNG files to have a high-resolution gradient. New gradients can be added in the folder with the same format: it must be horizontal! The height of the image does not matter.

2. **Terrain images:** They are loaded in the same way as color maps. They can be 8bit or 16bit PNG images. (It is recommended to use 16bits and high-resolution images to create smooth terrains.) They must be square! The program is made for images with identical width and height. Grayscale images are more suited, since the height map is computed based on the luminosity of pixels, but colored images are supported.



Example terrain image: Mountains

The luminance of a pixel is computed using $L = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$.

3. **default_scene.toml**: TOML config file that defines what image is used for data visualization and what the default parameters value.

```
[data]
path = "resources/Terrains/Mountains-16.png"
render_size = 20.0
render_height = 20.0

[lights]
positions = [[8.0, 5.0, 0.0]]
colors = [[1.0, 1.0, 1.0]]

[camera]
look_at = [0.0, 0.0, -1.0]
rotations = [0.0, 0.0, 0.0]
dist = 10.0

[gradient]
paths = ["resources/ColorMaps/inferno.png",
         "resources/ColorMaps/magma.png",
         "resources/ColorMaps/plasma.png",
         "resources/ColorMaps/rainbow.png",
         "resources/ColorMaps/viridis.png"]
```

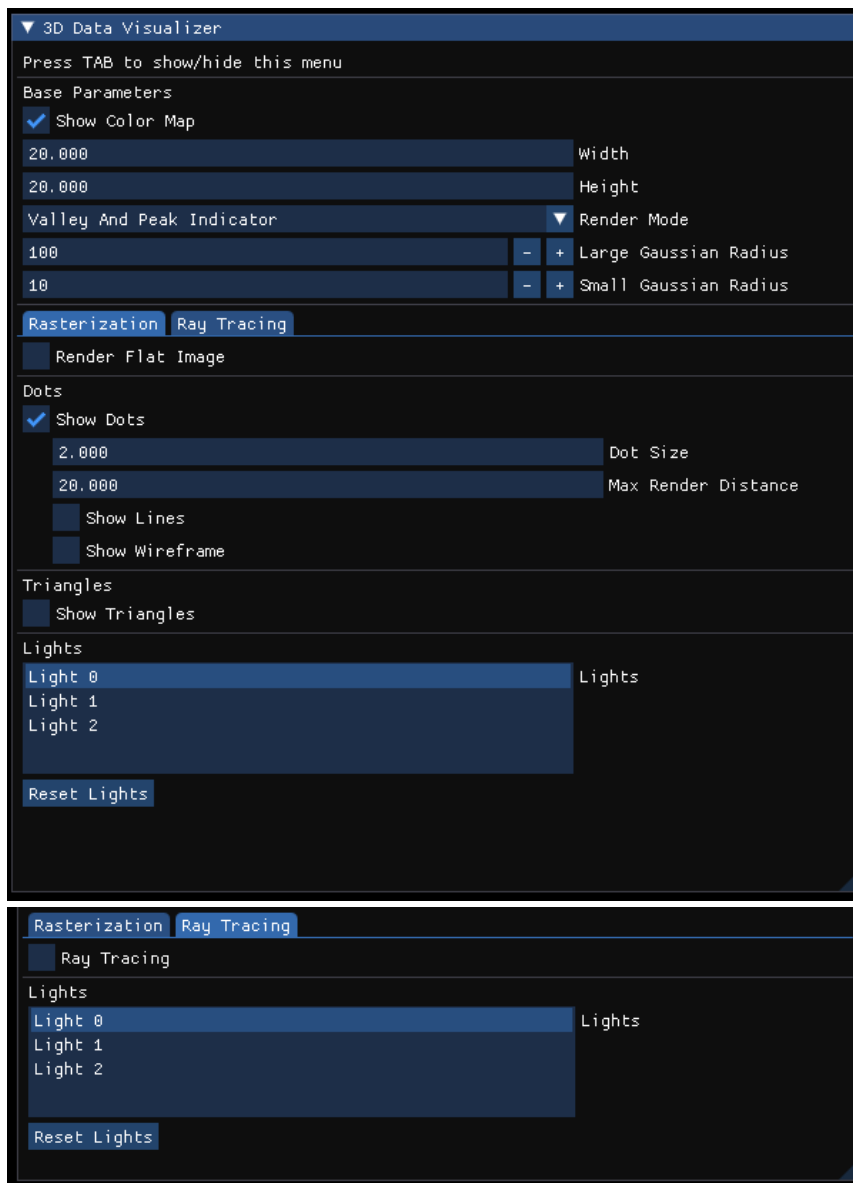
This is the list of parameters configurable in the TOML file.

- image path to use for the data visualization
- rendering size and height for the data
- list of light positions and colors
- camera position and orientation
- list of image paths to use as color map gradients.

II. Usage

A. User Interface

The UI allows the user to choose which feature is enabled and to customize the available parameters. When enabling a feature, the UI shows the parameters that can be modified, which were otherwise hidden. It has 2 main tabs: Rasterization and Raytracing.



B. Keyboard Shortcuts

Key	Action
TAB	Show/Hide UI menu.
H	Show help.
C	Apply the next color map.
L	Place the light source at the current camera position.
Shift + L	Add an additional light source at the current camera position.
Down Arrow	Select the next light source.
Up Arrow	Select the previous light source.
Del	Delete the selected light source.
N	Clear all light sources and reinitialize to a default light.
R	Add 0.1 to the red channel of the selected light.
G	Add 0.1 to the green channel of the selected light.
B	Add 0.1 to the blue channel of the selected light.
Shift + R	Subtract 0.1 from the red channel of the selected light.
Shift + G	Subtract 0.1 from the green channel of the selected light.
Shift + B	Subtract 0.1 from the blue channel of the selected light.

III. Code Outline

A. Helper Files

1. utils.h (general utils file)

This file contains the definition of the program structs and enums, to store 16bit color pixels, images and light sources.

```
// Dot Rendering Mode Enum
enum class RenderingMode {
    FixedColor = 0,
    Gradient = 1,
    Slope = 2,
    Edges = 3
};

// Pixels and Image Data
struct Pixel {
    unsigned short R = 0;
    unsigned short G = 0;
    unsigned short B = 0;
};

// Image Data
struct ImageData {
    std::vector<Pixel> pixels;           // The image pixel data
    int width = 0;                     // The image width (in pixels)
    int height = 0;                    // The image height (in pixels)
    float renderSize = 1.0f;           // The dot cloud render width (from the TOML config)
    float renderHeight = 1.0f;         // The dot cloud render height (from the TOML config)
};

// Light
struct Light {
    glm::vec3 position;
    glm::vec3 color;
};
```

It also contains the global variables of the program, that cannot be changed through the UI menu.

```
// GLOBAL VARIABLES
const int WIDTH = 1200;               // Program window width
const int HEIGHT = 800;               // Program window height
const int MAX_LIGHT_AMOUNT = 20;      // Max amount of light sources in the scene
ImageData imageData;                  // image data for the 3D visualization
std::vector<ImageData> colorMaps;      // list of color maps images
int activeColorMap = 0;                // index of the color map currently in use
Window* WINDOW;                       // Pointer to the GL window
Trackball* TRACKBALL;                 // Pointer to the camera trackball
```

This file also has helper functions to load the default config and relevant image files.

```
static glm::vec3 tomlArrayToVec3(const toml::array* array){ ... }

static std::vector<Pixel> loadPixelsFromImage(const char* filePath, int& width, int& height){ ... }

static void readInitialConfig(Trackball* trackball, ImageData& image, std::vector<ImageData>& colorMaps, std::vector<Light>& lightList)
```

2. gui.h (GUI utils file)

This file contains all parameters and variables that can be controlled from the UI.

```
// General
bool showGui = true;           // Show the menu
bool showRaytracingTab = false; // Whether the rasterization or ray tracing tab is active
float renderHeight = 5.0f;     // Current data render height
float renderSize = 20.0f;      // Current data render width
int edgeSmallBlurSize = 10;    // Small gaussian radius for edge detection
int edgeLargeBlurSize = 100;   // Large gaussian radius for edge detection
bool showColorMap = true;      // Show the active color map in the corner of the screen

// Quad
bool showFlatQuad = false;     // Show the image as a flat plane
float height = 0.0f;           // Height at which the flat plane is rendered

// Dots
bool showDots = false;         // Show the dot cloud
bool showLines = false;        // Show the vertical lines
bool showWireframe = false;    // Show the wireframe
float dotSize = 2.0f;          // Dot render size
float maxRenderDistance = 20.0f; // Maximum distance at which dots should be visible
RenderingMode renderMode = RenderingMode::FixedColor; // Render mode

// Triangles
bool showTriangles = false;    // Show triangles between vertices

// Lights
std::vector<Light> lights{};    // List of light sources
size_t selectedLightIndex = 0;  // Index of the currently selected light

// Ray Tracing
bool showRaytracing = false;    // Show the data using ray tracing
int maxSteps = 100;             // Amount of ray tracing steps
bool useBinarySearch = false;   // Whether binary search is used to get a cleaner result
```

gui.h also contains the function rendering the UI and some UI-related helper functions for the help menu, the lights, the color maps, debugging and keyboard shortcuts.

```

static void printHelp() { ... }

static void resetLights() { ... }

static void selectNextLight() { ... }

static void selectPreviousLight() { ... }

static void deleteLight() { ... }

// Prints the time elapsed during the start and end points
static void printElapsedTime(std::chrono::time_point<std::chrono::high_resolution_clock> start,
                             std::chrono::time_point<std::chrono::high_resolution_clock> end,
                             const char* text) { ... }

static void loadNextColorMap() { ... }

// Key Pressed Handler
static void keyPressedHandler(int key, int /* scancode */, int action, int /* mods */) { ... }

// Draws the UI menu
static void renderGUI() { ... }

```

3. glutils.h (OpenGL utils file)

This file contains references to all the OpenGL objects (arrays, buffers, textures, etc.) used throughout the program, as well as some useful constants to render quads and triangles on the screen.

```

// Indices needed to render a rectangle as 2 triangles
const unsigned int QUAD_INDICES[] = {
    0, 1, 2, // First Triangle
    2, 3, 0  // Second Triangle
};

// On-screen positions of the raytracing image plane
const std::vector<float> RAYTRACING_VERTICES = {
    // Positions      // Texture Coords
    -1.0f, -1.0f, 0.0f, 0.0f, 0.0f, // Bottom-left
    1.0f, -1.0f, 0.0f, 1.0f, 0.0f, // Bottom-right
    1.0f, 1.0f, 0.0f, 1.0f, 1.0f, // Top-right
    -1.0f, 1.0f, 0.0f, 0.0f, 1.0f // Top-left
};

// On-screen positions of the corners of the color map rectangle
const std::vector<float> COLORMAP_VERTICES = {
    // Positions      // Texture Coords
    -0.95f, -0.95f, 0.0f, 0.0f, 0.0f, // Bottom-left
    -0.75f, -0.95f, 0.0f, 1.0f, 0.0f, // Bottom-right
    -0.75f, -0.90f, 0.0f, 1.0f, 1.0f, // Top-right
    -0.95f, -0.90f, 0.0f, 0.0f, 1.0f // Top-left
};

```

```

GLuint dotVAO, dotVBO;
GLuint wireframeVAO, wireframeVBO, wireframeEBO;
GLuint triangleVAO, triangleVBO, triangleEBO;
GLuint raytracingVAO, raytracingVBO, raytracingEBO;
GLuint lightUBO;
GLuint lightVAO, lightVBO;
GLuint quadVAO, quadVBO, quadEBO;
GLuint colormapVAO, colormapVBO, colormapEBO;
GLuint edgeDetectionVAO, edgeDetectionVBO, edgeDetectionEBO;
GLuint heightMapTexture, normalMapTexture, colorMapTexture, edgeMapTexture;

int dotAmount, wireframeVerticesAmount, triangleVerticesAmount;
std::vector<glm::vec3> vertices;
std::vector<glm::vec3> normals;
glm::vec3 aabbMin, aabbMax;

// Shaders
Shader lightShader;
Shader dotShader;
Shader quadShader;
Shader triangleShader;
Shader lineShader;
Shader raytracingShader;
Shader gaussianBlurShader;
Shader edgeDetectionShader;
Shader colormapVisualizationShader;

```

This file also contains some basic OpenGL helper functions to initialize the shaders, create textures from images and normal maps, as well as computing the vertex position corresponding to a specific pixel, based on its position and luminance value, with respect to the rendering width and height. It also has functions to handle the loading of vertices and indices into OpenGL arrays and buffers.

```

static void initializeShaders() { ... }

static GLuint createTexture(const ImageData& data) { ... }

static GLuint createNormalTexture(const ImageData& data, const std::vector<glm::vec3>& normalMap) { ... }

static float getHeightFromPixel(Pixel pixel) { ... }

static glm::vec3 getVertexFromPixel(const ImageData& data, int x, int z) { ... }

static void clearAndLoadNewVertices(const std::vector<glm::vec3>& vertices, GLuint* VAO, GLuint* VBO) { ... }

static void clearAndLoadNewVerticesAndIndices(const std::vector<glm::vec3>& vertices, const std::vector<unsigned int>& indices, GLuint* VAO, GLuint* VBO, GLuint* EBO) { ... }

```

B. Main File

The main file (main.cpp) contains the functions initializing the OpenGL objects and buffers, computing the vertices and triangles, and rendering the data. Each feature will be explained in detail later. The main function has the following structure:

- Create OpenGL window object

- Read initial config
- Compute list of vertices, and setup buffers and arrays
- Compute height, normal and edge maps and create textures
- Create OpenGL buffer for lights
- While loop:
 - Update window, inputs and clear screen viewport
 - Check if rendering dimensions changed in the UI and recompute vertices and the normal and edge maps.
 - Compute MVP
 - Create a UBO for the lights
 - Bind all the relevant textures
 - For each feature, bind the shader and render
 - Show buffer to screen
- Clear the buffers

C. GLSL Shader Files

The Excel spreadsheet is a detailed overview of the shader files, with their inputs, outputs and descriptions. It also contains the list of the shader programs used, with their stages.

IV. Detailed Features

A. Flat Quad

```
static void loadQuadVertices(GLuint& VAO, GLuint& VBO, GLuint& EBO, const std::vector<float>& vertices) { ... }

static void loadFlatQuadVertices(const ImageData& data, GLuint& quadVAO, GLuint& quadVBO, GLuint& quadEBO) {
    float offsetX = -0.5f * data.renderSize / static_cast<float>(data.width);
    float offsetZ = -0.5f * data.renderSize / static_cast<float>(data.height);
    std::vector<float> vertices = {
        // Positions                                     // Texture Coords
        -0.5f * data.renderSize + offsetX, 0.0f, -0.5f * data.renderSize + offsetZ, 0.0f, 0.0f, // Bottom-left
        0.5f * data.renderSize + offsetX, 0.0f, -0.5f * data.renderSize + offsetZ, 1.0f, 0.0f, // Bottom-right
        0.5f * data.renderSize + offsetX, 0.0f, 0.5f * data.renderSize + offsetZ, 1.0f, 1.0f, // Top-right
        -0.5f * data.renderSize + offsetX, 0.0f, 0.5f * data.renderSize + offsetZ, 0.0f, 1.0f // Top-left
    };
    loadQuadVertices(quadVAO, quadVBO, quadEBO, vertices);
}
```

To render the data image as a flat plane, 2 functions are used. The function **loadQuadVertices** loads the given vertex data to a VAO, VBO and EBO. The data is a float array containing the world position and texture position of the 4 corners of the quad. The function **loadFlatQuadVertices** computes the world position of the flat image corners, then calls the **loadQuadVertices** function to load them to a VAO, VBO and EBO.

```
glBindVertexArray(quadVAO);
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
glBindVertexArray(0);
```

The quad is then rendered in the main loop as 2 triangles.

B. Dots

```
static std::vector<glm::vec3> loadVertices(const ImageData& data, glm::vec3& aabbMin, glm::vec3& aabbMax) { ... }
```

The dot cloud is generated from the image data as a list of dot positions, with one position for each pixel. The function also computes the bounding box of the entire cloud. Each time the render dimensions are changed in the UI, this list is regenerated.

```
vertices = loadVertices(imageData, aabbMin, aabbMax);
dotAmount = imageData.width * imageData.height;
clearAndLoadNewVertices(vertices, &dotVAO, &dotVBO);
```

In the main function, the list of vertices is created, then loaded into a VAO and VBO.

```
glPointSize(dotSize);
glBindVertexArray(dotVAO);
glDrawArrays(GL_POINTS, 0, dotAmount);
```

The dots are then rendered as GL_POINTS in the main loop.

C. Lines

The lines are rendered in the same way as the dots, but with the line shader instead of the dot shader. For each vertex (x, y, z) , the shader creates a new vertex on the XZ plane $(x, 0, z)$ and draws a line between them instead of just rendering the dot. In C++, the exact same data structures are used: dotVAO and dotVBO.

D. Wireframe

```
static int loadWireframeVertices(int width, int height, const std::vector<glm::vec3>& vertices,
                                GLuint* wireframeVAO, GLuint* wireframeVBO, GLuint* wireframeEBO) { ... }
```

The wireframe is generated from the list of vertices from the dot cloud. It uses an EBO to efficiently define the lines forming the wireframe using the indices of the vertices. The function loads the vertices into a new VAO and VBO, then uses the EBO to define the lines composing the wireframe: for each dot, a line is created to its right neighbor and its top neighbor. (if they exist) The function returns the number of vertices indices to render.

```
glBindVertexArray(wireframeVAO);
glDrawElements(GL_LINES, wireframeVerticesAmount, GL_UNSIGNED_INT, 0);
```

The wireframe is rendered as GL_LINES using the indices in the EBO.

E. Normal Map

```
static std::vector<glm::vec3> computeNormalMap(const ImageData& data, const std::vector<glm::vec3>& vertices) { ... }
```

The normal map is computed as a list of normal vectors, for each vertex position. It is obtained by computing the derivative in the x and z direction at each position. A texture is then created using the util function **createNormalTexture**, in the **glutils.h** file.

The normal map is used in the triangle rendering and the raytracing to compute the diffuse lighting and the slope gradient.

F. Edge Detection Map

```
static GLuint createEdgeMapTexture(const ImageData& data, const int smallGaussianFilterSize, const int largeGaussianFilterSize){ ... }
```

The edge map is computed using gaussian filters shaders and texture ping-ponging. The function **createEdgeMapTexture** takes the image and the filter size and outputs an OpenGL texture. It has the following outline:

- Create 5 frame buffers (FBOs) and 5 textures
- In FBO 0: apply the small gaussian filter horizontally from the original image
- In FBO 1: apply the small gaussian filter vertically from the image in FBO 0
- In FBO 2: apply the large gaussian filter horizontally from the original image
- In FBO 3: apply the large gaussian filter vertically from the image in FBO 2
- In FBO4: subtract the images in FBO 3 and FBO 1
- Create a copy of the texture of FBO 4
- Clear the FBOS and their textures
- Return the copy texture

The edge map texture is then used in the triangle and raytracing shaders to visualize the edges on the terrain.

G. Light UBO

```
// Light UBO
glGenBuffers(1, &lightUBO);
glBindBuffer(GL_UNIFORM_BUFFER, lightUBO);

// Allocate memory for 20 Lights (with padding so vec4 is used)
glBufferData(GL_UNIFORM_BUFFER, 2 * MAX_LIGHT_AMOUNT * sizeof(glm::vec4), nullptr, GL_DYNAMIC_DRAW);

// Bind the UBO to binding point 0
glBindBufferBase(GL_UNIFORM_BUFFER, 0, lightUBO);
```

The triangles and raytracing shaders allow for up to 20 dynamic light sources. They use a UBO to have a dynamic number of lights as uniform shader parameters.

```
static void loadLightsToUBO(const std::vector<Light>& lightArray, GLuint UBO){ ... }
```

In the main loop, the function **loadLightsToUBO** is called to load the light colors and positions in the UBO.

H. Shader Rendering Mode

The program can render 3 different visualizations : dots/lines/wireframe, triangles and raytracing. They can use different rendering modes:

- Fixed Color: Everything is rendered with a single color (with diffuse lighting for triangles and raytracing)
- Height Gradient: The color depends on the height.
- Slope Gradient: The color depends on the steepness of the slope
- Edge Map Gradient: The color depends on the edge map, generated from the data.

```
vec3 computeColorAtPos(vec3 pos){
    // Compute the surface color based on the render mode
    vec3 surfaceColor;
    switch(mode){

        case 1: // Mode 1 : gradient based on height.
            surfaceColor = computeGradient(pos.y, 0, renderHeight);
            break;

        case 2: // Mode 2 : gradient based on slope steepness
            vec3 normal = getTerrainNormalAtPos(pos);
            float value = 1.0 - abs(dot(normal, vec3(0, 1, 0)));
            surfaceColor = computeGradient(value, 0.0, 1.0);
            break;

        case 3: // Mode 3 : Different colors for peaks and valleys
            surfaceColor = computeGradient(getEdgeAtPos(pos).y, 0.2, 0);
            break;

        default: // Mode 0 : Fixed color
            surfaceColor = computeGradient(0.5, 0, 1);
            break;
    }

    // Compute diffuse lighting for all lights and output final color.
    vec3 finalColor = vec3(0);
    for (int i = 0; i < lightAmount; i++){
        finalColor += computeDiffuseLighting(lightColor[i].xyz, surfaceColor, pos, lightPos[i].xyz);
    }
    return finalColor;
}
```

The fragment shaders for the triangles and raytracing use the function **computeColorAtPos** to determine the color that should be rendered at a given position. The way the color is computed varies depending on the rendering mode.

The fragment shader for the dots/lines uses a similar function, but without diffuse lighting: it uses a darkening gradient when the dots are rendered far away. The maximum render distance can be modified in the UI.

I. Triangles

```
static int loadTriangleVertices(int width, int height, const std::vector<glm::vec3>& vertices,  
                               GLuint* triangleVAO, GLuint* triangleVBO, GLuint* triangleEBO) { ... }
```

The vertices used to render the triangles are generated from the dot cloud vertices. The vertices are stored in a VAO and VBO. Then, for each square in the grid, the indices of the 4 corner vertices are stored in an EBO.

```
glBindVertexArray(triangleVAO);  
glDrawElements(GL_LINES_ADJACENCY, triangleVerticesAmount, GL_UNSIGNED_INT, 0);  
glBindVertexArray(0);
```

The vertices are then rendered 4 by 4 as GL_LINES_ADJACENCY. For each 4 vertices, the shader will create 2 triangles and render them.

J. Raytracing

```
loadQuadVertices(raytracingVAO, raytracingVBO, raytracingEBO, RAYTRACING_VERTICES);
```

For the raytracing, the program creates a quad covering the entire screen, and the shader computes a ray for each pixel. The quad vertices are stored in a constant list in the **glutils.h** file.

```
glBindVertexArray(raytracingVAO);  
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);  
glBindVertexArray(0);
```

It is rendered exactly like the flat quad.

```
void computeRayAttributes(vec2 TexCoords, out vec3 origin, out vec3 direction) { ... }  
vec3 rayMarching(vec3 origin, vec3 direction, float tNear, float tFar, out bool hit) { ... }
```

The fragment shader computes the position on the terrain hit by the ray from the input texture coordinate. (representing the pixel position on the screen) The ray is computed in the **computeRayAttributes** function. Then, the function **rayMarching** is used to find the position where the ray hits the terrain. Once the position is determined, the function **computeColorAtPos** is used to compute the final color, similarly to the triangle shader.

K. Binary Search Raytracing

```
vec3 rayBinarySearch(vec3 pos1, vec3 pos2, bool isUnder)
```

If the binary search option is activated, the ray marching in the shader performs an additional step. Once the terrain is crossed, the function **rayBinarySearch** is used to determine the exact hit position more accurately.